Inside RTAL SYNTECH

> **V1.0.2 update:** Earlier disclaimers stating that final source had not yet been reviewed are superseded by the static audit of the uploaded V1.0.0 FINAL ZIP. Historical benchmarks remain unverified without runtime logs.

High-Quality Sound with an Efficient DSP Architecture

Technical architecture and performance analysis — RTAL SYNTECH V1.0.0 FINAL
Document revision: 1.0.2 (source-audited)
Date: 11 November 2026
Platform: ESP32-S3 / JC4827W543 (MAIN)
Sampling rate: 32,000 Hz | **Block size:** 256 audio frames | **Released configuration:** 6-note polyphony

> **Scope and evidence:** This document combines the architecture publicly documented by Joakim Eriksson for Scintilla with observations from the RTAL SYNTECH development and hardware-test sessions. Historical numbers are labeled as such. The release configuration (32 kHz / 256 frames / six notes) is drawn from the V1.0.0 FINAL project documentation and development records. This is not a line-by-line audit of the packaged firmware, and it is not an independent, reproducible benchmark of the final release. Measured timings on different chips, sounds, active voices, and effect settings are **not directly comparable**.

Executive summary

RTAL SYNTECH brings an Elka Synthex-inspired voice architecture to a compact ESP32-S3 touch instrument. The fundamental design lesson from Joakim Eriksson's **Scintilla** is that sound quality and embedded efficiency can coexist when the audio hot path is written to avoid needless work: integer/fixed-point processing, lookup tables, selective parameter preparation, tailored modulation paths, and disciplined allocation of memory and CPU time.

The most useful design lessons are **algorithmic cost control** and **memory locality**. A historical RTAL discussion reported a major speedup after moving a frequently accessed reverb buffer from PSRAM to internal SRAM. However, the final V1.0.0 SRAM-first implementation has now been **confirmed in the source**; the historical benchmark log has not been independently verified. The previously stated reverb timings are therefore presented below only as *unverified development notes*, not as proven release performance.

1. Origins and attribution

Joakim Eriksson developed Scintilla, an integer-C implementation of an Elka Synthex-style engine. The public Scintilla repository documents a voice engine with **two PolyBLEP oscillators, noise, five filter modes based on zero-delay feedback, two envelopes, two LFOs, glide, oscillator sync, ring modulation and cross modulation**. Its synthesizer layer handles layering, voice modes, chorus, reverb and a limiter. The project also explains how audio results are compared to its `synthex-web` reference implementation.

Primary upstream references:

- Joakim Eriksson: https://github.com/joakimeriksson
- Scintilla project: https://github.com/joakimeriksson/scintilla-fm1
- Earlier ESP32 / ESP32-P4 experiments: https://github.com/joakimeriksson/esp32

Credit: Scintilla and Joakim's prior Synthex-related work provided the core technical foundation and major inspiration for RTAL SYNTECH. RTAL-specific GUI, MIDI integration, preset storage, board

port, real-time scheduling, adaptations and performance work must be distinguished from the upstream contribution rather than presented as an original invention of the DSP core.

Licensing note: The public `scintilla-fm1` repository is marked GPL-3.0. It also describes code provenance from `synthex-web` and other components. A redistribution of derived code requires a file-level licensing and attribution review against the **exact upstream revision copied into RTAL SYNTECH**, retaining relevant notices, license texts and source-distribution obligations. This document is explanatory, **not a substitute for a license audit**.

2. What makes the engine efficient?

2.1 Integer / fixed-point audio arithmetic

The engine is designed around integer-C DSP rather than a general-purpose floating-point synthesizer. Fixed-point arithmetic can make per-sample operations predictable and avoid repeated floating-point conversions or expensive transcendental functions. The actual advantage depends on the processor, compiler, numerical format and saturation requirements; fixed point is **not automatically faster for every operation**.

Design principle: keep the sample loop small, explicit and bounded; choose fixed-point ranges carefully so that overflow, clipping and coefficient precision remain under control.

2.2 Lookup tables instead of repeated expensive mathematics

Upstream `engine/sc_tables.h` contains pitch, cutoff, sine and log/exp-related tables generated for **44.1 kHz in that upstream version**. This is **not** SYNTECH V1.0.0's operating rate (32 kHz). Precomputed tables trade memory space for fewer repeated mathematical operations in real-time rendering.

Implication for SYNTECH: table design, interpolation, cache locality and memory placement can matter more than nominal CPU clock speed. In a 32-kHz port, sample-rate-dependent tables, oscillator phase increments, filter coefficients and time constants must be generated or adapted for 32 kHz; the final source confirms the main 32-kHz adaptations, though full numerical verification of all LUT coefficients remains pending.

2.3 Avoiding work for inactive voices

Polyphonic engines scale with the number of voices actually doing DSP work, not just the advertised maximum. Inactive-voice skipping avoids oscillators, envelopes and filters consuming cycles after the voice has become silent. The final V1.0.0 `sc_render()` implementation demonstrably skips fully idle voices.

Design principle: voice allocation, release tails, effect sends and note-off behavior must be considered together; a voice cannot simply be skipped while its audible release remains active.

2.4 Parameter preparation rather than indiscriminate recalculation

The Scintilla architecture separates parameter-dependent preparation from per-sample synthesis. During our RTAL investigations we identified `sc_reprep()` as an important preparation path and discussed consolidating redundant recalculations when numerous touch or MIDI events arrive.

A **coefficient update cadence** is an optimization tradeoff, not a blanket rule that every parameter is recomputed only once per audio block. Fast envelope sweeps, oscillator interactions, and audio-rate modulation can require more frequent work. The current upstream Scintilla documentation specifically notes that rapid filter-envelope attacks can trigger coefficient recalculation at nearly every sample in demanding passages.

2.5 Specialized render paths

The upstream engine distinguishes ordinary paths from harder cases such as per-sample oscillator cross-modulation, PWM, synchronization and fast filter changes. This helps explain why a simple sustained chord may cost far less than a patch with aggressive modulation. **Polyphony alone is an insufficient performance metric.**

2.6 Real-time division of responsibilities

In RTAL SYNTECH the audio rendering thread is protected from expensive UI, touch, MIDI and SD-card operations as far as practical. A FreeRTOS command queue provides a boundary between event producers and the audio engine. This improves scheduling discipline, but it does not eliminate CPU pressure: processing a burst of parameter updates can still create costly blocks and possible XRUNs.

Scheduling principle: do not perform unpredictable filesystem I/O, allocations, display rendering or debug printing inside the audio-critical callback.

3. Why memory location matters: the reverb example

A reverb delay array discussed during development had this footprint:

```
int32_t buf[2048][8];   // 2048 × 8 × 4 = 65,536 bytes = 64 KiB
```

This calculation establishes only the **size of that example array**. It does not prove that the same object, type, dimensions or allocation are present in V1.0.0 FINAL. A frequently accessed delay buffer can be much more sensitive to PSRAM latency and cache misses than an infrequently accessed preset table.

Historical discussion — **unverified**	Reported figure
Reverb stage before relocation	approx. 500–600 µs
Reverb stage after relocation	approx. 154–182 µs
Relative reduction implied by the quoted ranges	approx. 64–74% (depending on paired values)

Evidence boundary: These numbers were repeated in the 4 November 2026 discussion. The measurement logs, test conditions, implementation change and whether the optimization survived into V1.0.0 have not been inspected here. They must **not** be published as a confirmed A/B benchmark of the released firmware. In particular, this document cannot conclude that memory placement alone caused the reported improvement.

Practical lesson: inspect frequently accessed buffers, verify their actual memory region in the map file / heap diagnostics, and benchmark with identical patches and build settings. Internal SRAM is limited and also needed for DMA-capable buffers, stacks and other real-time data.

4. Real-time timing budget — final configuration

The audio-block deadline follows directly from the actual final settings:

```
sample rate = 32,000 samples/s
block size  = 256 frames
T_block = 256 / 32,000 s = 0.008 s = 8.000 ms = 8,000 µs
```

8,000 µs is the wall-clock block interval, not a guaranteed 8,000-µs DSP allowance. I²S/DMA servicing, task scheduling, interrupts and other work require a safety margin. Whether all processing is included in a measured render time depends on the firmware's measurement boundaries.

4.1 Historical readings from *earlier* builds

A hardware log of **V0.4B** was discussed with **44.1 kHz and 128-frame blocks** (then an interval of about **2,902 µs**). It included roughly **174–176 µs** at low load, up to **421 µs** during active processing and a cited **665 µs peak**, with zero XRUNs and queue drops in that run. These figures describe that historical build only; they **must not** be recast as measurements of the final 32-kHz/256-frame release.

Other voice-scaling figures previously printed in this document (332 / 816 / 1,464 / 2,836 µs for 1 / 4 / 8 / 16 voices) are **removed** because a reliable matched test log, build configuration and voice-count methodology were not established. They cannot support a polyphony claim.

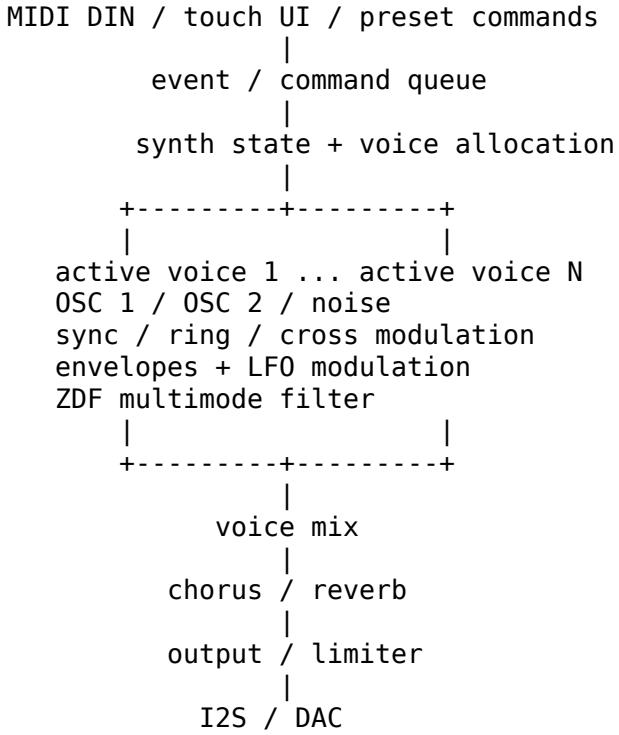
4.2 Upstream benchmarks are not ESP32-S3 benchmarks

Joakim's public `scintilla-fm1` README reports instruction-cost estimates for **256-frame blocks at 44.1 kHz on the FM-1 platform**, using its own hardware/emulator assumptions and selected patches. They demonstrate substantial variation with modulation and attack behavior. They cannot be transferred as percentages or render times to RTAL SYNTECH on ESP32-S3.

A previously circulated **~80 µs ESP32-P4** figure lacks a reproducible patch/voice/effect/test-scope definition here, so it is **not retained as a verified benchmark**. ESP32-P4 and ESP32-S3 also have materially different processor and memory systems.

5. Synthesizer topology and cost drivers

A conceptual processing diagram:



This is a simplified architectural diagram; exact task ownership and DSP execution order are documented in the accompanying source audit.

The most demanding presets tend to combine several of the following conditions: many simultaneously active voices or layers; cross-modulation and synchronization; continuously variable oscillator waveforms; fast filter-envelope sweeps; fast LFO modulation; and active effects. Consequently, worst-case validation must test **patch complexity as well as note count**.

6. RTAL-specific engineering around the engine

The SYNTECH project integrates the synthesis engine into a touch-enabled ESP32-S3 instrument rather than simply running Joakim's original UI. The V1.0.0 project context includes:

Area	RTAL SYNTECH V1.0.0 context
Main controller	ESP32-S3, JC4827W543 touch board
Audio	32 kHz, 256 audio frames per block, 32-bit I2S path
Target polyphony	6 notes on MAIN
Touch display	NV3041A QSPI, 480 × 272, LVGL 8.4 / GT911
DIN MIDI	RX GPIO17, TX GPIO18, 31,250 baud
MIDI control	MOPHO banked CC mode (CC32), standard direct-CC mode, MIDI Learn
Preset handling	128 program positions on SD: 000–079 factory, 080–127 user; `.rtal`
Real-time integration	Dedicated audio work and queued UI/MIDI commands

These are project design / release-context descriptions, **supported where indicated by a static source-code audit, but not by an independent hardware benchmark**.

7. Performance engineering checklist

For any future DSP change, the following evidence should be captured on the same hardware and firmware build:

1. Build identifier, CPU frequency, compiler flags and memory layout.
2. Sample rate, block size and exact voice/layer configuration.
3. Preset, oscillator interactions, filter mode, effects and modulation depth.
4. Average, maximum and high-percentile block render times over a sufficiently long run.
5. XRUN, queue overflow, clipping and MIDI-event-loss counters.
6. Rapid parameter-change tests separately from steady-state chord tests.
7. Cold-start, SD preset-load and GUI interaction tests.
8. An A/B comparison changing **one variable at a time**.

Suggested CPU headroom measure:

```
headroom ≈ 1 - (measured_peak_render_time / 8000 µs)
```

A negative result indicates a block overrun. A positive result is only an approximation of available margin: it excludes overhead not captured by the measured interval.

8. What we can responsibly conclude

Supported architecture: Eriksson's Scintilla is deliberately designed as a compact integer-C synthesizer with lookup tables, PolyBLEP oscillators and performance-aware render paths. The upstream documentation describes measurable complexity differences between presets.

Historical RTAL development discussion (not verified for final release): A large reverb speedup was reported after an SRAM-versus-PSRAM change. The SRAM-first implementation is now confirmed in the final source, but without comparable historical logs the **quantitative improvement remains unverified**.

Release settings: The RTAL SYNTECH V1.0.0 MAIN-board project is a six-note ESP32-S3 instrument at 32 kHz / 256 audio frames. Unverified higher-voice and ESP32-P4 speed claims do not establish additional playable notes on the release.

Not established by this document: a reproducible final-firmware worst-case latency sweep; 8/16-note support on the released ESP32-S3; a definitive causal analysis of every compiler or DSP optimization; or exact file-by-file license compliance in the final distribution.

9. Credits and further reading

Original synthesis engineering and inspiration: Joakim Eriksson — Scintilla / Synthex-related projects.

RTAL integration and subsequent engineering: RealTimeAudioLab — ESP32-S3 platform adaptation, touch interface, MIDI modes, SD preset system, scheduling and the RTAL performance-investigation work documented here.

References:

1. Joakim Eriksson, *Scintilla for M-VAVE FM-1*, https://github.com/joakimeriksson/scintilla-fm1 — project README, engine description, numerical comparisons, instruction-count benchmarks and GPL-3.0 license.
2. Joakim Eriksson, *ESP32 experiments*, https://github.com/joakimeriksson/esp32 — additional historical platform context.
3. RTAL SYNTECH development discussions, **30 October and 4 November 2026** — historical hardware readings and an *unverified* memory-placement discussion, as qualified above. V1.0.0 FINAL release configuration from the 11 November 2026 project documentation.

Documentation revision 1.0.1 (corrected against V1.0.0 FINAL configuration) — 11 November 2026. This text is intended for GitHub publication as a technical companion document, with numerical claims explicitly scoped to their evidence.

V1.0.2 addendum: Final firmware source audit

A **static source audit** of `RTAL_SYNTECH_V1_0_0_FINAL(2).ZIP` was performed on November 11, 2026. It establishes:

- **32,000 Hz / 256 stereo frames / 8,000 µs:** `RTAL_AUDIO_SAMPLE_RATE`, `FRAMES`, `SC_FS`, and the I²S configuration agree, with a compile-time assertion between DSP and I²S rates.
- **32-kHz DSP port:** `sc_tables.h` explicitly identifies regenerated pitch/cutoff tables. LFO scaling, envelope timing, 96-sample filter-coefficient blocks, chorus, reverb delay/decay and limiter timing are adapted to 32 kHz.
- **Actual signal path:** Core-1 audio task (priority 23), FreeRTOS command queue, batched parameter preparation, active-voice rendering, optional chorus and shared reverb, limiter, Q15 to 32-bit I²S output conversion.
- **Reverb memory optimization is present:** The **64-KiB reverb ring is allocated in internal SRAM first**, falling back to PSRAM only if necessary. A claim that FINAL always stores the ring in PSRAM is incorrect. The boot diagnostic `[FIX5 MEMORY]` tells which memory was actually used.
- **Memory allocation:** Synthesizer state also prefers internal SRAM with PSRAM fallback. Chorus delay (8,192 B), limiter delay (1,536 B payload), scratch/audio buffers and the LVGL buffer (57,600 B) are identifiable in source. Exact full RAM/flash accounting requires a linker map and runtime measurements.
- **Polyphony terminology:** The engine allocates **eight voice slots per layer**, not exactly six total. SYNTECH's six-note operating target should not be confused with the underlying slot capacity or two-layer DSP instances.

Limits: This was not a target-device compilation, hardware benchmark or complete numerical LUT verification. Historical reverb A/B timing claims therefore remain unverified by this ZIP. See `RTAL_SYNTECH_V1_0_0_FINAL_Quellcode_Audit_DSP_Speicher_32k_DE.md` for the detailed German source audit.